

Lecture 36: Audio-Language Models II

How a model speaks

It answered out loud

You asked a question. About **200 milliseconds** later a voice started answering, with the right emphasis in the right places. You interrupted halfway through, and it stopped.

It did not call a speech-synthesis service. The same network that chose the words produced the sound.

Last lecture got sound *into* a language model. This one is the harder direction, and it needs an idea we have been avoiding since ch9.

The contradiction

What a language model does

Picks the next item from a **finite vocabulary**, by a softmax over maybe 100,000 choices. Discrete, one at a time.

What sound is

A **continuous** pressure curve, arriving 16,000 numbers a second, where the exact values matter and there is no alphabet.

This is the same collision as L34, where a continuous diffusion model met a discrete language model over images. The resolution is the same too, and it is audacious:

Invent a vocabulary for sound. Then talking is just next-token prediction.

Outline

A vocabulary for sound

Semantic tokens and acoustic tokens

Generating sound left to right

Talking at the same time

A vocabulary for sound

What if a sound were made of words? The first three attempts all fail, and each one fails for a reason worth knowing.

Attempt 1: just round the samples

Each sample is a number. Round it to one of 256 levels and you have a vocabulary of 256. This is literally what WaveNet did.

The vocabulary is fine. 256 is tiny. With a logarithmic spacing that matches loudness perception, 8 bits per sample sounds acceptable.

The sequence length is fatal. You are back to 16,000 tokens per second - exactly the problem L35 opened with. Nothing was gained.

We do not need a smaller *vocabulary*. We need fewer *tokens*.

Attempt 2: quantize frames instead of samples

Push the audio through an encoder that reduces it to a few frames per second, then snap each frame's vector to the nearest entry of a learned codebook. This is **vector quantization**, exactly as in L34, and the audio becomes a grid of integers.

And now the vocabulary is fatal. To match a real codec's quality with a single codebook you would need roughly

$$2048^8 = 2^{88} \approx 3 \times 10^{26} \text{ entries.}$$

That is not a large softmax. That is more entries than there are stars in the observable universe - by at least a factor of a hundred, on any published estimate.

Attempt 1 had too many tokens. Attempt 2 has too many *words*. The fix is to stop choosing.

Attempt 3: residual vector quantization

Residual Vector Quantization (RVQ) is the idea that made audio tokenizable, and it is three lines long.

1. Quantize the frame with codebook 1. Keep the index.
2. Look at what is **left over** - the error you just made.
3. Quantize *that* with codebook 2. Repeat for 8 codebooks.

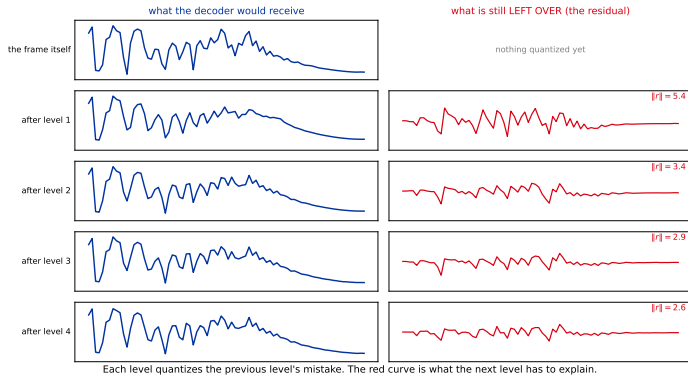
Formally, with $r_0 = x$ and $e_k(\cdot)$ the nearest entry in codebook k :

$$r_k = r_{k-1} - e_k(r_{k-1}), \quad \hat{x} = \sum_{k=1}^8 e_k(r_{k-1})$$

Eight codebooks of 2,048 give $8 \times 11 = 88$ bits per frame - the same expressive power as 2^{88} entries, from 16,384 vectors you can actually store.

The cost: a frame is now **8 tokens, not 1**. Section 7 is entirely about what to do about that.

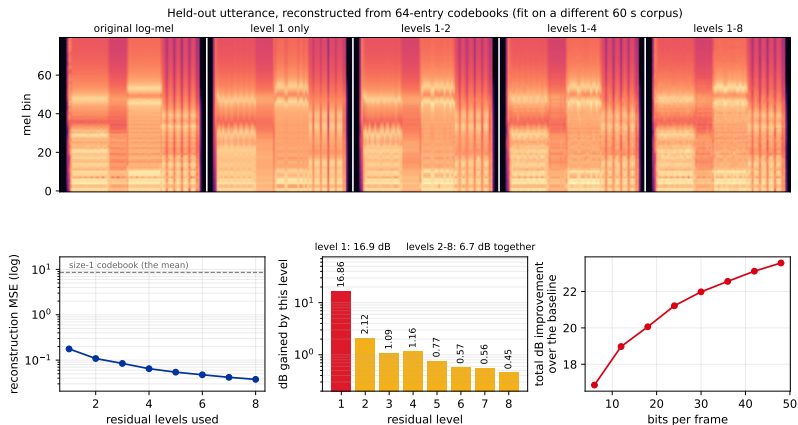
The same idea, watched level by level



One real frame, quantized four times. **Left:** what the decoder would receive so far. **Right:** the error still outstanding, which is the next level's entire job.

Notice the residual is not just *smaller* - it is **less structured**. Level 1 takes the big obvious shape; by level 4 what remains looks close to noise. That is exactly why the returns diminish so fast on the next slide.

What does each level actually buy?

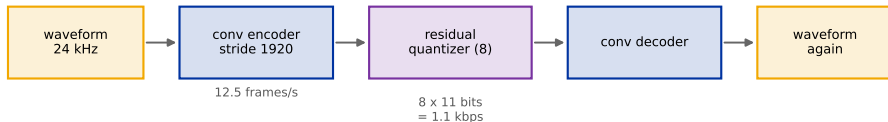


Stand-in, not a codec: k-means on log-mel frames, which cannot be turned back into sound. Read the shape, not the numbers.

Sharply front-loaded: level 1 buys about $2.5\times$ what all seven remaining levels add together, for one eighth of the bits. *That ratio* is the finding; the dB values belong to this stand-in, not to a real codec. Same shape as L34's image codebooks - which is why level 1 is the one everybody fights over.

The codec, and where it actually operates

Waveform in, waveform out - NOT a spectrogram, because a spectrogram cannot be played back (L35).



Trained end to end with a multi-scale spectral loss plus discriminators - not squared error.

On raw waveforms - not on the mel spectrograms we spent six slides building. A spectrogram cannot be played back (L35, the phase slide), so a system that must *emit sound* cannot live there. Mel is for models that only listen.

The codecs worth knowing

Encoder, residual quantizer, decoder, trained end to end.

On raw waveforms - not on the mel spectrograms we spent six slides building. A spectrogram cannot be played back (L35, the phase slide), so a system that must *emit sound* cannot live there. Mel is for models that only listen.

Codec	Who, when	Note
SoundStream	Zeghidour et al., 2021	introduced residual quantization here
EnCodec	Défossez et al., 2022	good enough for music, not just speech
Mimi	Défossez et al., 2024	the one Moshi runs on

Mimi: 24 kHz in, **12.5 frames per second** out - a **1,920×** cut in sequence length - with **8** codebooks of 2,048, which is **1.1 kbps**.

(The released checkpoint ships 32 quantizers; Moshi uses 8 of them.)

Two things that make the training work

Rounding has no gradient

"Snap to the nearest entry" is a step function: its derivative is zero everywhere it is defined. The encoder would never learn.

Straight-through estimator. On the backward pass, pretend the quantizer was the identity and pass the gradient through unchanged. Exactly the L34 trick, same justification, and it still feels like cheating.

The loss is not squared error

Minimising squared error on a waveform gives muffled audio: two signals can be perceptually identical and numerically far apart, because the ear does not hear phase.

Instead: **multi-scale spectral loss** (compare log-spectrograms at several window sizes) plus **adversarial discriminators** - the ch8b idea, doing exactly what it is good at, judging whether something sounds real.

The second point is why a codec at 1.1 kbps sounds better than a 1990s codec at ten times the bitrate.

Is residual quantization the only way?

No, and a student who remembers L34 should already be suspicious - we saw there that growing a codebook has sharply diminishing returns, and that the field escaped by changing the *quantizer* rather than its size.

Lookup-free and scalar quantization. Drop the learned codebook; quantize each dimension to a few fixed levels and let the combination be the code. The audio analogue of MAGVIT-2 from L34.

Single-codebook tokenizers. If one frame is one token, Section 7's whole problem disappears. The price is quality, and the race is to push the frame rate down without paying it.

Same tension as every compression slide in this course: fewer tokens, or better sound.

Semantic tokens and acoustic tokens

What was said, and how it sounded. Separating them is the difference between a codec and something a language model can reason with.

The goal: split what from how

A language model built on codec tokens has a problem. The tokens encode *sound*, and most of what makes a sound is not what was said - it is the voice, the room, the microphone.

What we want. Level 1 carries *what was said*. The rest carry *who said it and how*. Then the model can reason over level 1 like it reasons over text, and treat the others as decoration.

Kyutai's demonstration of the finished system makes it vivid: keep level 1, replace the rest, and you hear **the same words in a completely different voice**.

Note the word **goal**. Nothing so far makes this happen.

And it does not happen on its own

Take a plain residual quantizer that nobody told about meaning, and resynthesize audio from **level 1 alone**. If level 1 were "semantic", the words would survive and the voice would not.

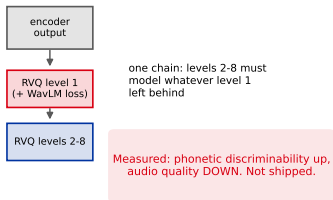
The opposite happens. Probing EnCodec this way (Zhang et al., 2024), level 1 alone retains a **speaker similarity of 0.92** - the voice comes through almost perfectly - while the content is destroyed.

Which makes sense in hindsight, and our own measurement two slides ago said the same thing from the other direction: **level 1 is whatever explains the most variance**, and in speech that is the gross spectral shape of a particular voice. Nothing in the training objective ever mentioned words.

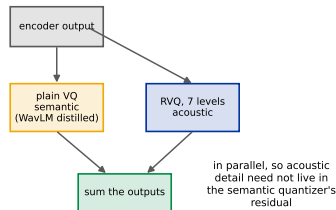
"Coarse" and "semantic" are not synonyms.

Mimi's fix, and the attempt that failed first

The obvious design: distill semantics into level 1



What Mimi ships: a split



They tried the obvious thing, measured it, and restructured (Défossez et al., 2024)

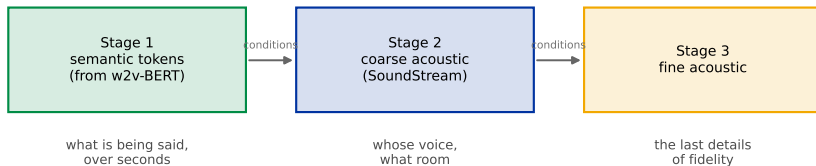
The obvious repair is to *teach* level 1 to be semantic, with a loss making it match **WavLM** - the self-supervised speech model from L35, whose features track *what was said*. Mimi's authors tried exactly that: it made the first quantizer much better at telling phonemes apart, and **made the audio worse**, because whatever level 1 stopped encoding still had to be carried by the residual it handed on.

So they took it out of the chain. **Worth copying as a way of working:** the obvious design was tried, measured, and restructured - and the paper reports the failure.

AudioLM: three stages, three jobs

AudioLM (Borsos et al., 2022) took the same split and made it the architecture.

Each stage is its own transformer, conditioned on everything before it



One model doing all three spends its capacity in the wrong place: gorgeous audio that says nothing.

Why split it at all? Coherence over ten seconds and fidelity over ten milliseconds are different problems. One model doing both spends its capacity in the wrong place - which is precisely what the sample-level models in L35 did.

The same three stages, in detail

Stage	Tokens	Job
1	semantic, from w2v-BERT (L35)	long-range structure: syntax, meaning, where the melody is going
2	coarse acoustic, from SoundStream	voice, recording conditions
3	fine acoustic	the last details of fidelity

Each stage is its own transformer, conditioned on the output of the one before.

Why split it at all? Coherence over ten seconds and fidelity over ten milliseconds are different problems. One model doing both spends its capacity in the wrong place - it produces audio that sounds gorgeous and says nothing, which is precisely what sample-level models did in L35.

Generating sound left to right

We have a vocabulary. Now the awkward part: there are eight tokens per timestep and a language model emits one at a time.

Eight tokens per frame, one tape

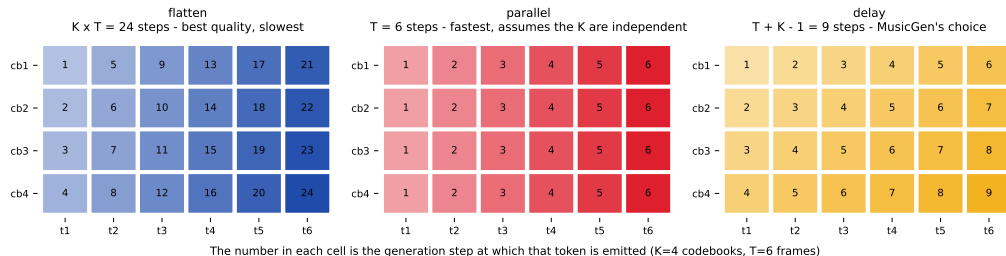
Residual quantization gives a **grid**: 8 codebooks deep, T frames wide. Autoregressive generation needs a **line**. Somebody has to choose the order, and the choice is not free.

Read it in the wrong order and you ask the model to predict fine detail before it has committed to the coarse content it is detailing.

Read it too carefully and you pay 8 forward passes per frame. At 12.5 Hz that is 100 steps per second of audio - real time evaporates.

Exactly the raster-order question from L34, one dimension deeper.

Three orders, three prices



Flatten - best quality, and $K \times T$ steps. Every token sees every earlier one.

Parallel - T steps, and it assumes the 8 codebooks of a frame are independent given the past. They are not.

Delay - offset each codebook by one step. $T + K - 1$ steps, and codebook k can still see codebook $k - 1$ for its own frame.

MusicGen (Copet et al., 2023) uses delay with $K = 4$: $T + 3$ steps instead of $4T$, for nearly the quality of flattening.

VALL-E: speech synthesis as next-token prediction

VALL-E (Wang et al., 2023) trained a language model on codec tokens over 60,000 hours. Text in, codec tokens out, decode to audio. Nothing about it is specific to speech - it is GPT with a different vocabulary.

The striking part is what came free. Prefix the generation with **3 seconds** of somebody's voice as codec tokens, and the model continues *in that voice* - because continuing the prompt is the only thing it knows how to do. **Zero-shot voice cloning with no fine-tuning**, from a capability nobody added on purpose.

The output is codec tokens, so getting audio back is just running the codec's decoder - the point the next slide but one makes in general.

Three seconds of anyone's voice is a phone call, a voicemail, a video. The impersonation problem is a direct consequence of the architecture, not a misuse of it.

The technical mitigation is **watermarking**: embed a signal in generated audio that survives compression and re-recording. Moshi ships one.

The other lineage: do not generate left to right at all

Everything so far has been autoregressive. A large part of speech synthesis went the other way, and you have seen this argument before.

	Autoregressive	Non-autoregressive
How	next token, one at a time	denoise the whole utterance at once
On what	discrete codec tokens	continuous latents (no quantization)
Examples	VALL-E, CosyVoice	F5-TTS, MaskGCT, Voicebox
Strength	prompting, streaming	speed, and no tokenizer to lose fidelity in

This is L34 section 7, again, for a different sense. Discrete-and-autoregressive versus continuous-and-diffusion, the same trade-off, the same unresolved argument. When the same fork appears twice in two lectures, it is telling you something about the shape of the problem rather than about images or audio.

Getting back to a waveform

Two lineages, and they answer "how do I turn this into sound" differently.

Mel plus a vocoder (the Tacotron era). The model predicts a mel spectrogram; a separate network - HiFi-GAN and relatives - invents the phase that L35 threw away and produces a waveform. Two models, trained separately.

Codec tokens plus the codec's decoder (now). There is no separate vocoder, because **the codec decoder is the vocoder** - it was trained as the other half of the encoder that made the tokens.

A common confusion worth heading off: in a codec system, asking "and then which vocoder?" has no answer. The decoder already is one.

By hand: one second of audio, both directions

	Listening (L35)	Speaking (L36)
raw samples	16,000	24,000 (Mimi runs at 24 kHz)
frames	100 log-mel	12.5 codec frames
after the encoder	50 Whisper vectors	12.5 frames $\times$ 8 codebooks
tokens the model handles	25 (pooled)	100
bits	continuous vectors	$12.5 \times 88 = 1,100$ bits

Speaking costs more than listening. Understanding can pool aggressively and throw away anything not needed for meaning. Generation has to produce every bit that comes out of the loudspeaker.

1,100 bits per second is **1.1 kbps** - the number from the codec slide, now derived rather than quoted.

Talking at the same time

The one problem in this chapter that has no counterpart anywhere in vision.

A conversation is not turn-taking

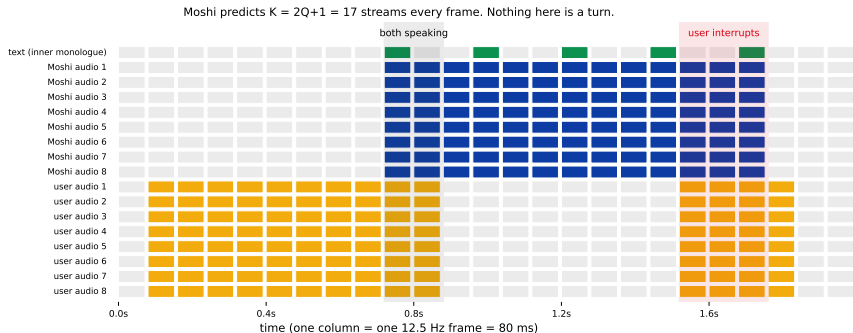
Everything built so far assumes turns: you speak, it listens, you stop, it answers. Real conversation is not like that. A system that can listen and speak *simultaneously* is called **full duplex**, borrowing the term from telephony.

- ▶ You say "mhm" while the other person is still talking
- ▶ Both of you start at once, and one yields
- ▶ You interrupt as soon as the answer is wrong
- ▶ The pause before "yes" carries the real answer

A turn-based model cannot do any of this **by construction**, however fast it gets. The voice-activity detector from L35 is a *guess* about when you finished, and it is making that guess with no idea what you were saying.

An image arrives all at once. A conversation is two streams that never stop. This is where the audio problem stops resembling the vision problem.

Moshi: predict everything, every frame

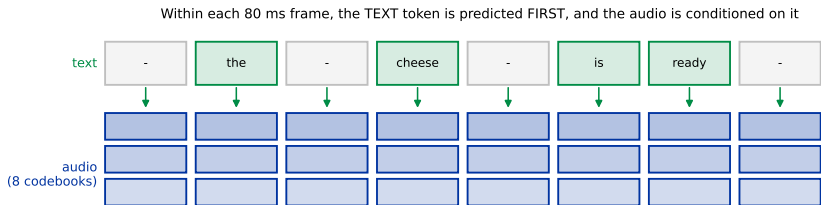


Moshi (Défossez et al., 2024) drops turns entirely. Every 80 ms it predicts $K = 2Q + 1 = 17$ streams: 8 codebooks of its own audio, 8 of yours, and 1 text token.

It is **always** listening and **always** speaking. Silence is not the absence of output, it is a prediction like any other - which is what makes an interruption something the model can simply respond to, rather than an exception some surrounding code must handle.

Inner monologue

A speech-to-speech model trained only on audio tokens speaks fluently and reasons badly. All that text pretraining is in there somewhere, and generating sound does not reach it.



So the language model's fluency reaches the speech - and shifting this stream turns the same model into a recogniser or a synthesiser.

The text stream is an *internal scaffold*, not a stage in a pipeline. Nothing is transcribed and re-synthesized.

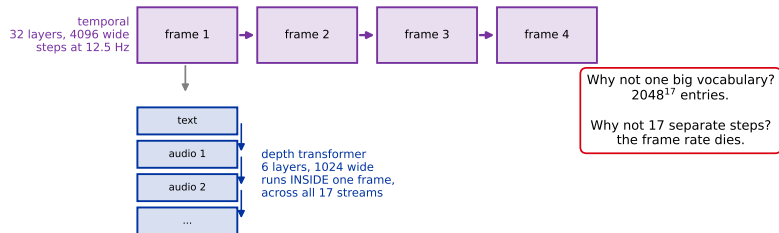
What that buys

Moshi's answer: for each frame, predict the **text** token *first*, then that frame's audio tokens conditioned on it. The model writes what it is about to say, then says it, 12.5 times a second.

Keeps the linguistic quality of a language model while the output stays genuinely speech-to-speech.
Nothing is transcribed and re-synthesized - the text is an internal scaffold, not a stage in a pipeline.

And it comes with a bonus: shift the text stream relative to the audio and the same model does **streaming recognition** or **streaming synthesis**.
Three systems, one set of weights.

Why it takes two transformers



The expensive model runs only 12.5 times a second. The cheap one runs 17 times inside each of those steps.

The two transformers, spelled out

Temporal transformer. 32 layers, 4,096 wide. Steps once per frame, at 12.5 Hz. This is where the language ability lives, and it does the expensive work only 12.5 times a second.

Depth transformer. 6 layers, 1,024 wide. Runs *inside* one frame, across the 17 streams, so each stream sees the ones decided before it. Small enough that 17 of these cost little.

This structure is the **RQ-Transformer** (Lee et al., 2022), borrowed from image generation rather than invented here.

Result: **160 ms** of theoretical latency, about **200 ms** measured. For scale, one Mimi frame is 80 ms, so the floor is a small number of frames - though the paper reports the 160 ms rather than breaking it down, so do not quote a decomposition.

The landscape, mid-2026

Model	Year, who	Weights	Notable for
Moshi / Mimi	2024, Kyutai	open	full duplex, inner monologue, 12.5 Hz
Qwen3-Omni	2025, Alibaba	open	one model for text, image, audio, video
GPT-realtime	2024–, OpenAI	closed	the product that made this mainstream
Gemini native audio	2024–, Google	closed	32 tokens/s billing, very long context

Worth noticing: unlike the text-model race, the open-weights entries here are genuinely at the front. Moshi published the architecture, the codec, and the failed ablations - which is why this lecture could be built at all. Almost nothing about the closed systems is documented well enough to teach.

What is still unsolved

- ▶ **Memory.** A one-hour conversation is 45,000 frames and 765,000 tokens. Nobody has a good answer.
- ▶ **More than two speakers.** The 17 streams assume exactly one of you.
- ▶ **Evaluation.** S2SBench for reasoning loss, MMAU and AIR-Bench for audio understanding - all young, none decisive.
- ▶ **Impersonation.** Watermarking helps; it does not solve it.

The open question worth watching: does learning to speak cost you thinking?

S2SBench measures a real drop when models are trained for speech output. Qwen3-Omni claims *no* degradation against its single-modality counterparts.

Both are published. They disagree. That is the honest state in 2026, and it is a better thing to tell students than a tidy answer.

Recap

- ▶ A language model needs a **finite vocabulary**. Rounding samples gives too many tokens; one big codebook needs 3×10^{26} words.
- ▶ **Residual quantization** solves it by quantizing the leftovers, over and over. Our stand-in measurement: level 1 buys roughly **2.5**× what levels 2-8 add between them - the ratio is the lesson, not the decibels.
- ▶ Codecs work on **waveforms**, not spectrograms, because a spectrogram cannot be played back.
- ▶ Level 1 is **not** semantic for free - it keeps the voice and loses the words. Mimi runs a semantic quantizer **in parallel**, after the obvious fix hurt the audio.
- ▶ Eight tokens per frame need an order: **flatten, parallel, or delay**.
- ▶ **Full duplex** is the genuinely new problem. Moshi predicts 17 streams every 80 ms and never takes a turn.

Next: `ml/llm_training/`. Every model here was a pretrained language model with a new sense bolted on - so the question left is how that language model got made.